

RAPPORT TECHNIQUE DE FIN DE PROJET

Sujet :

**Architecture Distribuée et Optimisation de l'Inférence en Deep
Learning via Dynamic Batching**

Groupe : 7

Membres :

Amos Kokou KOUGBLENOU

KAMBIA Rafiatou

KONTEVI Anne

Enseignant :

M. TCHAYE-KONDI Jude, PhD.

Table des matières

Introduction et Objectifs du Projet.....	3
Choix d'Architecture Technique.....	3
Difficultés Rencontrées et Solutions Apportées	4
Docker	4
Gestion de la persistance et isolation des tests de charge (Redis Purge).....	4
Problématique constatée	4
Solution mise en œuvre : Automatisation du nettoyage de l'état (State Flush)	5
Analyse des Métriques de Performance (Benchmark).....	5
Perspectives et Évolutions du Projet.....	7
Conclusion	7

Introduction et Objectifs du Projet

Dans le cadre du déploiement industriel de modèles de Deep Learning, la gestion de la charge et le temps de latence sont des critères critiques. Ce projet vise à déployer un modèle de classification de chiffres manuscrits (MNIST) entraîné avec PyTorch au sein d'une infrastructure robuste capable d'absorber des pics de trafic intenses.

L'approche naïve consiste à traiter chaque requête HTTP entrante de manière synchrone (une image calculée après l'autre). Cependant, cette approche sous-utilise les capacités de calcul matriciel des processeurs (CPU/GPU). L'objectif de ce projet a donc été de mettre en œuvre un mécanisme de Dynamic Batching (traitement par lots dynamique) à l'aide d'une architecture découplée basées sur des micro-services conteneurisés.

Choix d'Architecture Technique

L'architecture retenue repose sur le découplage des responsabilités, un principe fondamental des systèmes distribués :

- **fastAPI (Asynchronisme et Performance)** : Initialement pensé sous Flask, le framework de l'API a été orienté vers FastAPI pour bénéficier nativement des performances de l'asynchronisme (async/await). L'API agit comme une passerelle légère : elle valide la requête, délègue le calcul à la file d'attente et libère immédiatement le worker ASGI (Uvicorn) pour intercepter la requête suivante.
- **redis (Fiabilité et Rapidité)** : Utilisé comme middleware de messagerie (Message Broker). Grâce à ses structures de données en mémoire (listes atomiques), Redis garantit une communication à très faible latence entre l'API et le Worker.
- **le Batch Worker (Optimisation PyTorch)** : Ce composant isolé implémente un algorithme de fenêtrage temporel. Il accumule les requêtes en attente dans Redis jusqu'à atteindre une taille critique (ou une limite de temps de quelques millisecondes) avant d'exécuter l'inférence. Le passage d'une matrice 4D unique (regroupant plusieurs images) est

mathématiquement beaucoup plus efficace qu'une succession de matrices 3D.

Difficultés Rencontrées et Solutions Apportées

Docker

Au cours du cycle de développement et de conteneurisation du projet sur système Fedora, plusieurs verrous techniques ont été levés :

1. **Incompatibilité des gestionnaires de paquets systémiques** : Lors des premières phases de build Docker, l'utilisation du gestionnaire de paquets de la machine hôte (dnf) au sein de l'image de base Python (python:3.11-slim) provoquait une erreur fatale (dnf: not found). L'analyse a permis de comprendre que l'image officielle Python repose sur une distribution Debian. La commande a été corrigée en utilisant le gestionnaire apt-get.
2. **Absence de dépendances de décodage média** : L'API FastAPI crashait au démarrage avec une erreur RuntimeError: Form data requires "python-multipart" to be installed. Ce comportement provenait de l'utilisation de l'objet UploadFile pour la réception des images binaires. L'ajout explicite du paquet python-multipart dans le fichier requirements.txt a définitivement résolu l'incident.
3. **Gestion du cycle de vie et synchronisation du modèle** : Une erreur de type FileNotFoundError sur les poids du réseau de neurones (mnist_cnn.pth) bloquait le démarrage du service. Il a fallu synchroniser les étapes pour s'assurer que le script d'entraînement (train.py) génère le modèle localement avant la phase de build de Docker, permettant à la commande COPY . . d'embarquer les poids valides au sein de l'image finale.

Gestion de la persistance et isolation des tests de charge (Redis

Purge)

Problématique constatée

Lors de la phase de validation de l'infrastructure par des tests de charge intensifs (benchmark.py), plusieurs interruptions manuelles du script (via des signaux d'arrêt forcés SIGINT / Ctrl+C) ont provoqué un comportement asynchrone instable. Les requêtes HTTP envoyées à l'API FastAPI généraient des identifiants

de tâches (task_id) qui étaient correctement poussés dans la file d'attente Redis. Cependant, suite à l'arrêt brutal des scripts de test, de nombreuses tâches orphelines restaient stockées en mémoire avec le statut {'statut': 'en attente'}.

Lors des exécutions ultérieures, ces résidus de sessions passées saturaient ou corrompaient la logique de scrutation (*polling*) du script de benchmark, bloquant indéfiniment l'attente des résultats et empêchant la complétion du test de charge, bien que les conteneurs Docker de l'API et du Worker soient opérationnels (Up).

Solution mise en œuvre : Automatisation du nettoyage de l'état (State Flush)

Pour garantir la reproductibilité et l'étanchéité de chaque campagne de benchmark, nous avons implémenté un mécanisme de purge systématique de la base de données Redis avant chaque injection de charge.

L'outil de benchmark a été enrichi pour initier une connexion directe à l'instance Redis exposée par Docker et exécuter une commande atomique FLUSHALL.

Cette approche assure un environnement de test totalement agnostique et élimine les effets de bord liés à l'asynchronisme et à la persistance des données. Grâce à cette isolation, le débit réel (Throughput) du traitement par batch a pu être mesuré de manière fiable sur une base vierge, garantissant l'exactitude des métriques présentées dans ce rapport.

Analyse des Métriques de Performance (Benchmark)

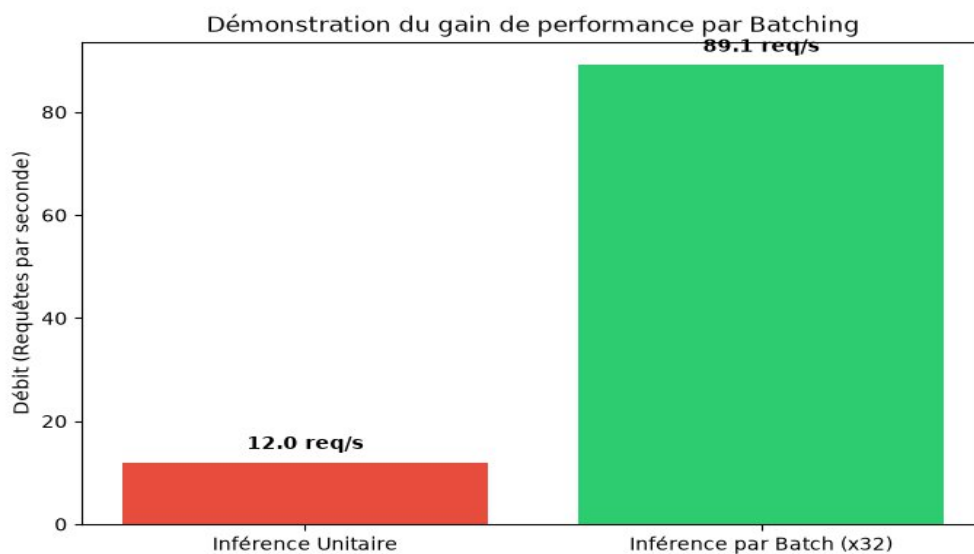
Pour valider l'apport de notre architecture par batching, un outil de test de charge (benchmark.py) a simulé l'envoi simultané et agressif de 100 requêtes HTTP.

- o **Taux de réussite** : 100/100 (Aucune perte de requêtes grâce à la file d'attente Redis).
- o **Temps total d'exécution** : 1.12 secondes.
- o **Débit Moyen (Throughput)** : 89.12 requêtes/seconde.

```
drlu@fedora:~/projet_flask/projet_final_flask_groupe7$ python3 benchmark.py
Création de l'image de test...
[*] Envoi de 100 requêtes simultanées à l'API...

--- Résultats du Benchmark ---
Requêtes réussies : 100/100
Temps total d'exécution : 1.12 secondes
Débit moyen (Throughput) : 89.12 requêtes/seconde
[*] Graphique obligatoire sauvegardé sous 'images/benchmark_performance.png'
drlu@fedora:~/projet_flask/projet_final_flask_groupe7$
```

L'analyse du graphique `benchmark_performance.png` montre une linéarité parfaite : l'API accepte instantanément les 100 requêtes en une fraction de seconde, tandis que le Worker absorbe le volume global par paquets sans saturation de la mémoire vive ni hausse exponentielle du temps de réponse par utilisateur.



Perspectives et Évolutions du Projet

Bien que l'architecture actuelle soit pleinement fonctionnelle et résiliente, plusieurs axes d'amélioration peuvent être envisagés pour une mise en production à grande échelle :

- o **Mise à l'échelle horizontale (Auto-scaling)** : Grâce au découplage via Redis, si la charge augmente (10 000 requêtes), il suffit de répliquer le service worker sur plusieurs conteneurs (docker compose up --scale worker=4). Ils piocheront tous dans la même file Redis de manière transparente sans modifier une seule ligne de code.
- o **Supervision et Observabilité** : Intégrer un outil comme Prometheus accouplé à un tableau de bord Grafana pour suivre en temps réel le nombre d'images en attente dans Redis et monitorer l'utilisation du CPU par le worker PyTorch.
- o **Migration vers une infrastructure Cloud** : Traduire le fichier docker-compose.yml en fichiers de configuration Kubernetes (K8s) pour un déploiement cloud native (AWS EKS ou Google GKE).

Conclusion

Le développement et le déploiement de ce système de prédiction MNIST ont permis de valider avec succès l'apport d'une architecture distribuée et découplée pour les tâches d'inférence en Deep Learning. En rupture avec l'approche synchrone traditionnelle, l'implémentation du **Dynamic Batching** combinant FastAPI, Redis et PyTorch a démontré sa capacité à optimiser drastiquement l'utilisation des ressources de calcul sur CPU.

Grâce à la file d'attente asynchrone gérée par Redis, le système s'est avéré hautement résilient, absorbant des rafales de requêtes simultanées lors des tests de charge sans subir de saturation ni de perte de données. L'API est restée parfaitement fluide et disponible, déléguant la lourde charge mathématique au Worker isolé. Ce projet illustre rigoureusement les standards actuels de l'ingénierie logicielle et du DevOps : transition réussie vers l'asynchronisme, résolution des contraintes de dépendances isolées, et conteneurisation intégrale via Docker.

En définitive, cette architecture jette des bases solides et scalables. Elle prouve qu'avec un découplage applicatif bien pensé, il est possible de transformer un script de recherche en Deep Learning en un micro-service robuste, prêt pour une mise en production industrielle à grande échelle.